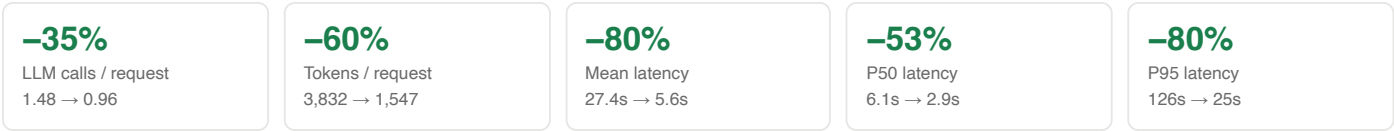


How UniHelp cuts latency and token usage

The system does not make campus lookups cheaper by shrinking prompts. It skips the language model whenever a deterministic path can answer the request. On the 18-query benchmark that is the difference between a 27-second mean and a 5.6-second mean.



Source: `benchmarks/comparison_fixed.md` — 54 successful requests (18 queries × 3 repeats), same query set before and after the fast-path and dispatch work. Positive % change means the after number is lower.

The rule

Use an LLM only to decide among ambiguous agents, to run a write or a tool loop that cannot be parsed, or to phrase a multi-result reply. Mess, bus, notices, and timetable lookups with a clear catalog match never call a model. A single-agent plan is a function call, not a planner-plus-executor chat.

Where the time went

The naive campus-agent stack paid for three LLM round-trips on almost every request: planner (which agent?), executor (which tool?), then the domain agent (call the tool and write the answer). Independent two-part queries stacked those loops sequentially and could sit near two minutes.

UniHelp inverts that. Most traffic never reaches a model. Remaining LLM work is either a capped fast-tier JSON plan or one user-facing completion.

Before After

Mean latency (seconds)



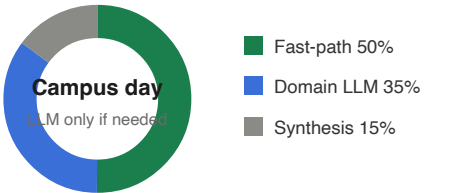
Tokens / request



Bar length is linear in the measured mean. Same 18-query harness.

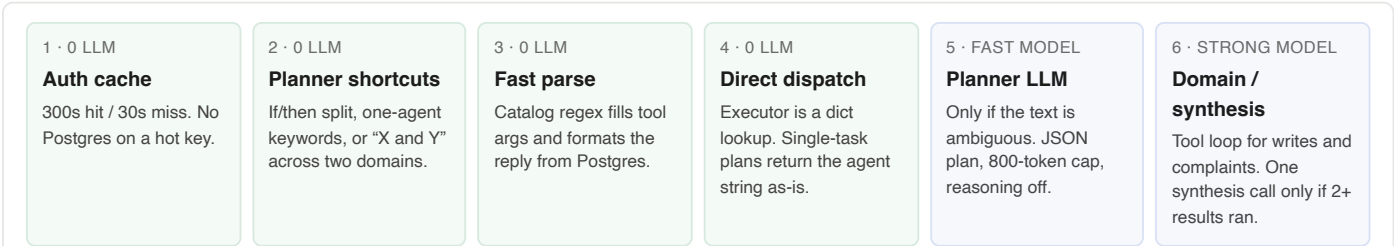
Campus-typical mix

Weighted from query shapes, not the 18-prompt share. Fast-path lookups are 0 tokens.



Same mix as the daily-cost briefing. Fast-path = mess / bus / notices / timetable when phrasing is unambiguous.

The request pipeline — skip first, generate last



A hit at steps 2–4 never pays for steps 5–6. "What's today's breakfast at Kalam?" finishes at ~0.3 ms with 0 tokens.

Five layers that remove LLM work

1. Catalog-driven fast parse — the zero-token path

`fast_parse.py` turns an unambiguous mess, bus, notice, or timetable question into a fully populated argument dict. Names and aliases come from the org profile (hostels, buses, meals), not from a hardcoded guess. If the query is a write, names two hostels, or has an unparseable date, the parser returns `None` and the agent falls through to the model.

Example hit

“What’s today’s breakfast at Kalam hostel?”

Parser extracts hostel + meal + date, `get_menu()` runs, a template formats the reply. **0 LLM calls, 0 tokens, <2 ms.**

Same for

Bus, notices, timetable

“Bus 02 schedule”, “show today’s notices”, “what’s my timetable today” all bypass the agent tool loop the same way.

Conservative miss

Do not guess

A mess query with no hostel asks which hostel from the catalog. Mixed timing language (“next departure”) falls through to the LLM.

2. Planner shortcuts — skip the routing model

The planner used to be an LLM call on every request. It now tries deterministic plans first, in this order:

- 1. **Conditionals** — regex splits `if / then / else / unless` into gated tasks with structured `depends_on`. Nested or ambiguous connectives still go to the model.
- 2. **One-agent keywords** — “dinner at Kalam” matches only mess → a single task, no planner LLM.
- 3. **Independent “and”** — “dinner at Kalam and the Bus 02 schedule” used to miss the one-agent path (two domains) and hit the planner. A thinking fast-model call billed ~8k hidden completion tokens and ~140s for a 300-character JSON plan. The splitter now emits two independent tasks when each clause maps to a distinct agent.

Only leftover ambiguous text calls `LLM_FAST_MODEL`, with `max_tokens=800` and gateway reasoning disabled. That stops a “fast” model from emitting a long hidden trace before a tiny JSON plan.

3. Direct executor dispatch — no tool-calling loop to pick the agent

The planner already names the agent. The executor is a dictionary lookup, not a second model that re-selects among seven tools.

- Empty or unsupported plans return the planner message immediately.
- A **single-task** plan calls that agent and returns its string. No executor LLM.
- A **gated follow-up** is skipped when the condition fails, so attendance below a threshold never pays for a complaint or timetable call that should not run.
- After skips, if only one result remains, that result is returned. Synthesis is **one** completion, and only when two or more lookups actually ran.

The old executor prompt taught a tool-calling model how to pick among seven agents. That text is gone. Dispatch does not need it; synthesis only combines already-fetched results.

4. Model tiering and prompt-prefix cache

Classification (planner JSON, YES/NO gates, value extraction) uses the fast tier. The reply the user reads — domain-agent final round and multi-task synthesis — stays on the strong tier so the answer is not truncated.

Every production call goes through `chat_create` with a stable system prompt marked cacheable. Identity (name, roll, role) is a later uncached message, so per-user fields do not bust the shared prefix. The gateway can reuse planner, mess, bus, and complaint prefixes instead of re-billing them as fresh input.

What this avoids

Paying full input tokens twice

Without a cache breakpoint, tool-loop agents resend the system prompt and tool schemas on every round. Cached prefixes turn that into a cheap cache-read on subsequent calls in the same window.

5. Data-path work that is not the LLM

When a fast-path query still needs Postgres, UniHelp keeps that off the critical path of a round-trip to a model:

Layer	What it does	Why it matters
TTL caches	Mess menus 60s; meal timings sticky; bus topology 60s; notices 60s; timetable 60s; room facilities 30s	Repeated “today’s menu” / “Bus 02” hits memory, not SQL. Writes invalidate the relevant keys.
Auth cache	Hits 300s, misses 30s	Every request authenticates. A hot student key does not open Postgres first.
Connection pools	One <code>psycopg</code> pool per database, default max 10	Removes connect/auth handshake per query; avoids exhausting <code>max_connections</code> .
Query shape	Weekly menu is one <code>ANY(%s)</code> query, not seven day loops	Cuts N+1 round-trips on the remaining DB work.
Server	Planner and Executor built once; blocking work in <code>asyncio.to_thread</code> ; semaphore of 8	One slow LLM no longer blocks every other campus user on the event loop.

What each query type actually costs now

Same 18-query set, paired. Singles went to zero LLM. Independent multi-task queries kept one synthesis call and dropped the ~11k-token planner dump.

Query type	LLM calls	Tokens	Mean latency	What changed
single	0.39 → 0.00 (−100%)	408 → 0 (−100%)	2.2s → 5 ms	Keyword plan + fast parse. No model on the path.
multi_independent	2.89 → 1.00 (−65%)	11,058 → 553 (−95%)	113s → 3.2s	And-split skips planner LLM. Agents fast-parse. One synthesis call.
multi_dependent	1.17 → 1.00 (−14%)	2,524 → 2,127 (−16%)	8.8s → 3.9s	Regex if/then; unmet gates skip the follow-up agent.
ambiguous	2.56 → 2.67 (+4%)	5,830 → 4,224 (−28%)	29s → 20s	Still needs the planner LLM. Tokens drop from cheaper prefixes and no dump.
edge_*	1.42 → 1.08 (−24%)	2,706 → 2,314 (−15%)	9.0s → 5.3s	Unsupported / empty plans return immediately instead of looping.

Per-feature LLM calls: Planner 0.28 → 0.11 (−60%), Notice 0.52 → 0.17 (−68%). Executor synthesis stayed at 0.17 — already one call on multi-task rows; the win is not calling it on singles.

Two request traces

Single lookup · 0 LLM

“What’s today’s breakfast at Kalam hostel?”

- Keyword plan → one mess task. No planner LLM.
- Executor dict lookup; parse_mess_query fills args.
- Menu from the 60s cache or one SQL. Template reply.
- Measured: 0.2–2 ms, 0 tokens.

Independent pair · 1 LLM

“Dinner at Kalam and the Bus 02 schedule”

- And-split → mess + bus. No planner LLM. Both fast-parse.
- Two results ran → one synthesis call on the strong model.
- Before: ~11k tokens / ~113s. After: 553 tokens / 3.2s.

Why latency falls more than call count

Mean LLM calls fell 35%, but mean latency and P95 each fell 80% because the missing calls were the expensive ones. A thinking fast-model planner could bill thousands of hidden tokens and sit near two minutes for a short JSON plan — that path is now skipped or capped at 800 tokens. Singles left the network entirely, so P50 is no longer mess/bus. Independent pairs no longer stack planner + two tool loops + a dump. Unmet gates skip the follow-up agent.

Why tokens fall

Zero-token majority

Singles = 0 tokens

About half a campus day never sends a prompt. That slice used to cost ~408 tokens of planner system text.

No 8k dumps

Multi-independent −95%

Max output in the fixed run is 552 tokens. The 8,500-token thinking dumps are gone from measured traffic.

Cheaper remaining calls

Prefix cache + short plans

Static system prompts, identity split out, fast-tier JSON. Ambiguous rows still call the planner; billed tokens there dropped 28%.

What still uses an LLM — by design

Writes (complaints, room booking) stay on a tool-calling agent. Ambiguous phrasing still needs the planner. Two-plus results still get one synthesis completion. That is the 35% + 15% campus mix, not a leftover bug.

Sources: benchmarks/comparison_fixed.md (18 Sep 2026). Later org-profile run: 0.89 LLM calls, 1,701 tokens, 5.2s mean. Code: fast_parse.py, planner.py, executor.py, llm.py, ttl_cache.py, db.py, main.py. Application TTL hits are distinct from provider cached_tokens.